

```

/*
=====
=====
    admin/admin-app.js
    -----
-----
    Wires together auth.js, db.js,
storage.js and youtube-fetch.js into a
    working Netflix-style dashboard:
navigation, CRUD forms, search, the
    drag-and-drop Weekly Top 100 manager,
settings, and overview charts.

=====
===== */

(() => {
    const DB = window.HVC.DB;
    const Storage = window.HVC.Storage;
    const YT = window.HVC.YouTubeFetch;

    // In-memory caches (refreshed from
Firestore on each view load / after writes)
    let songsCache = [];
    let artistsCache = [];
    let labelsCache = [];
    let newsCache = [];
    let top100Meta = { locked: false };
    let draggedRow = null;
    let pendingDelete = null; // { type, id }

```

```

let ytFetchResult = null;
let chartMostViewed = null;
let chartBreakdown = null;

/* =====
BOOTSTRAP ===== */

window.HVC.Auth.requireAdmin((user) => {
  document.getElementById("admin-
email").textContent = user.email;
  document.getElementById("admin-
avatar").textContent = (user.email || "A")
[0].toUpperCase();
  initNav();
  initModals();
  initSongForm();
  initArtistForm();
  initLabelForm();
  initNewsForm();
  initTop100();
  initSettingsForm();
  initSearch();
  document.getElementById("logout-
btn").addEventListener("click", () =>
window.HVC.Auth.logout());

  const mobileBtn =
document.getElementById("mobile-menu-btn");
  const sidebar =
document.querySelector(".sidebar");
  mobileBtn.hidden = false;
  mobileBtn.addEventListener("click", ()
=> sidebar.classList.toggle("is-open"));

```

```

        loadOverview();
    });

    /* ===== NAV
===== */

    function initNav() {
        const items =
document.querySelectorAll(".nav-item");
        items.forEach(btn => {
            btn.addEventListener("click", () => {
                items.forEach(b =>
b.classList.remove("is-active"));
                btn.classList.add("is-active");

document.querySelectorAll(".view").forEach(
v => v.classList.remove("is-active"));

document.getElementById(`view-${btn.dataset
.view}`).classList.add("is-active");

document.querySelector(".sidebar").classLis
t.remove("is-open");

                switch (btn.dataset.view) {
                    case "overview": loadOverview();
break;
                    case "songs": loadSongs(); break;
                    case "artists": loadArtists();
break;
                    case "labels": loadLabels();
break;
                    case "news": loadNews(); break;
                    case "top100": loadTop100();

```

```

break;
        case "settings": loadSettings();
break;
    }
    });
});
}

/* ===== MODALS
===== */

function initModals() {
    document.querySelectorAll("[data-
close]").forEach(btn => {
        btn.addEventListener("click", () => {

document.getElementById(btn.dataset.close).
hidden = true;
        });
    });
    document.querySelectorAll(".modal-
overlay").forEach(overlay => {
        overlay.addEventListener("click", (e)
=> {
            if (e.target === overlay)
overlay.hidden = true;
        });
    });
}

function openModal(id) {
document.getElementById(id).hidden = false;
}
function closeModal(id) {

```

```

document.getElementById(id).hidden = true;
}

function showToast(message, isError =
false) {
  const toast =
document.getElementById("toast");
  toast.textContent = message;
  toast.classList.toggle("is-error",
isError);
  toast.classList.add("is-visible");
  setTimeout(() =>
toast.classList.remove("is-visible"),
3000);
}

function confirmDelete(type, id, message)
{
  pendingDelete = { type, id };
  document.getElementById("confirm-
message").textContent = message;
  openModal("confirm-modal");
}

document.getElementById("confirm-delete-
btn").addEventListener("click", async () =>
{
  if (!pendingDelete) return;
  const { type, id } = pendingDelete;
  try {
    if (type === "song") { await
DB.deleteSong(id); await loadSongs(); }
    if (type === "artist") { await
DB.deleteArtist(id); await loadArtists(); }

```

```

        if (type === "label") { await
DB.deleteLabel(id); await loadLabels(); }
        if (type === "news") { await
DB.deleteNews(id); await loadNews(); }
        showToast("Deleted successfully.");
    } catch (err) {
        showToast(err.message || "Delete
failed.", true);
    }
    closeModal("confirm-modal");
    pendingDelete = null;
});

function fmtNum(n) {
    n = n || 0;
    if (n >= 1_000_000) return (n /
1_000_000).toFixed(1).replace(/\.0$/, "") +
"M";
    if (n >= 1_000) return (n /
1_000).toFixed(1).replace(/\.0$/, "") +
"K";
    return String(n);
}

function fmtDate(value) {
    if (!value) return "-";
    const d = value.toDate ? value.toDate()
: new Date(value);
    if (isNaN(d)) return "-";
    return d.toLocaleDateString("en-US", {
year: "numeric", month: "short", day:
"numeric" });
}

```

```

function escapeHtml(text) {
  const div =
document.createElement("div");
  div.textContent = text || "";
  return div.innerHTML;
}

/* =====
OVERVIEW ===== */

async function loadOverview() {
  const [songs, artists, labels, news] =
await Promise.all([
    DB.getSongs(), DB.getArtists(),
DB.getLabels(), DB.getNews(),
  ]);
  songsCache = songs; artistsCache =
artists; labelsCache = labels; newsCache =
news;

  document.getElementById("stat-
songs").textContent = songs.length;
  document.getElementById("stat-
artists").textContent = artists.length;
  document.getElementById("stat-
labels").textContent = labels.length;
  document.getElementById("stat-
news").textContent = news.length;
  document.getElementById("stat-
top100").textContent = songs.filter(s =>
s.weeklyRank).length;
  document.getElementById("stat-
featured").textContent = songs.filter(s =>
s.featured).length;

```

```

    // Recently added table
    const recent = [...songs].slice(0, 8);
    const tbody =
document.getElementById("overview-recent-
table").querySelector("tbody");
    tbody.innerHTML = recent.map(s => `
        <tr>
            <td></td>
            <td><div class="row-
title">${escapeHtml(s.title)}</div><div
class="row-sub">${escapeHtml(s.artistName
|| "")}</div></td>
            <td>${fmtNum(s.views)} views</td>
            <td>${fmtDate(s.addedAt)}</td>
        </tr>
    `).join("") || `<tr><td colspan="4"
class="empty-state">No songs yet.</td>
</tr>`;

    renderCharts(songs, { artists:
artists.length, labels: labels.length,
news: news.length });

    // Rough estimated YouTube quota usage
this session (1 unit per fetch call)
    const used =
parseInt(sessionStorage.getItem("hvc_admin_
yt_units") || "0", 10);
    document.getElementById("usage-bar-
fill").style.width = `${Math.min(100, (used
/ 10000) * 100)}%`;

```



```

        document.getElementById("usage-
label").textContent = `${used} / 10,000
units (this session)`;
    }

    function renderCharts(songs, counts) {
        const topViewed = [...songs].sort((a,
b) => (b.views || 0) - (a.views ||
0)).slice(0, 6);

        const ctx1 =
document.getElementById("chart-most-
viewed");
        if (chartMostViewed)
chartMostViewed.destroy();
        chartMostViewed = new Chart(ctx1, {
            type: "bar",
            data: {
                labels: topViewed.map(s => (s.title
|| "").slice(0, 18)),
                datasets: [{ data: topViewed.map(s
=> s.views || 0), backgroundColor:
"#e50914", borderRadius: 6 }],
            },
            options: {
                plugins: { legend: { display: false
} },
                scales: {
                    x: { ticks: { color: "#8a8a8a",
font: { size: 10 } }, grid: { display:
false } },
                    y: { ticks: { color: "#8a8a8a" },
grid: { color: "rgba(255,255,255,0.06)" }
},

```

```

    },
    },
  });

  const ctx2 =
document.getElementById("chart-breakdown");
  if (chartBreakdown)
chartBreakdown.destroy();
  chartBreakdown = new Chart(ctx2, {
    type: "doughnut",
    data: {
      labels: ["Songs", "Artists",
"Labels", "News"],
      datasets: [{
        data: [songs.length,
counts.artists, counts.labels,
counts.news],
        backgroundColor: ["#e50914",
"#f5c518", "#2ecc71", "#8a8a8a"],
        borderWidth: 0,
      }],
    },
    options: { plugins: { legend: {
position: "bottom", labels: { color:
"#c7c7c7", font: { size: 11 } } } } },
  });
}

/* ===== SONGS
===== */

async function loadSongs() {
  songsCache = await DB.getSongs();
  await ensureArtistOptions();

```

```

    renderSongsTable(songsCache);
  }

  function renderSongsTable(songs) {
    const tbody =
document.getElementById("songs-table-
body");
    document.getElementById("songs-
empty").hidden = songs.length > 0;
    tbody.innerHTML = songs.map(s => `
      <tr data-id="${s.id}">
        <td></td>
        <td><div class="row-
title">${escapeHtml(s.title)}</div></td>
        <td>${escapeHtml(s.artistName ||
"-")}</td>
        <td>${escapeHtml(s.label || "-")}
</td>
        <td>${escapeHtml(s.genre || "-")}
</td>
        <td>${fmtNum(s.views)}</td>
        <td>${s.weeklyRank ? "#" +
s.weeklyRank : "-"}</td>
        <td>
          ${s.featured ? '<span
class="badge badge-
featured">Featured</span>' : ""}
          ${s.trending ? '<span
class="badge badge-
trending">Trending</span>' : ""}
        </td>
        <td>${s.releaseDate ||

```

```

fmtDate(s.uploadedAt)}</td>
    <td>
        <div class="row-actions">
            <button class="edit-song-btn"
title="Edit"><svg viewBox="0 0 24 24"><path
d="M12 20h9M16.5 3.5a2.1 2.1 0 0 1 3 3L7
19l-4 1 1-4 12.5-12.5Z"
stroke="currentColor" stroke-width="1.6"
fill="none" stroke-linejoin="round"/></svg>
</button>
            <button class="delete-song-btn"
title="Delete"><svg viewBox="0 0 24 24">
<path d="M3 6h18M8 6V4a2 2 0 0 1 2-2h4a2 2
0 0 1 2 2v2m2 0v14a2 2 0 0 1-2 2H8a2 2 0 0
1-2 2V6h12Z" stroke="currentColor" stroke-
width="1.6" fill="none" stroke-
linecap="round"/></svg></button>
        </div>
    </td>
</tr>
` ).join("");

tbody.querySelectorAll(".edit-song-
btn").forEach(btn => {
    btn.addEventListener("click", (e) =>
openSongModal(e.target.closest("tr").datase
t.id));
});
tbody.querySelectorAll(".delete-song-
btn").forEach(btn => {
    btn.addEventListener("click", (e) =>
{
        const id =
e.target.closest("tr").dataset.id;

```

```

        const song = songsCache.find(s =>
s.id === id);
        confirmDelete("song", id, `Delete
"${song?.title}"? This cannot be undone.`);
    });
}

async function ensureArtistOptions() {
    if (artistsCache.length === 0)
artistsCache = await DB.getArtists();
    const select =
document.getElementById("song-artist");
    select.innerHTML = `<option value="">–
Select artist –</option>` +
        artistsCache.map(a => `<option
value="${a.id}" data-
name="${escapeHtml(a.name)}">${escapeHtml(a
.name)}</option>`).join("");
}

function initSongForm() {
    document.getElementById("add-song-
btn").addEventListener("click", () =>
openSongModal(null));

    document.getElementById("fetch-youtube-
btn").addEventListener("click", async () =>
{
        const url =
document.getElementById("song-youtube-
url").value.trim();
        const status =
document.getElementById("fetch-status");

```

```
        status.textContent = "Fetching...";
        status.className = "";
        try {
            ytFetchResult = await
YT.fetchVideoDetails(url);
            document.getElementById("song-
title").value = ytFetchResult.title;
            document.getElementById("song-
duration").value =
ytFetchResult.formattedDuration;
            document.getElementById("song-
views").value = ytFetchResult.views;
            document.getElementById("song-
likes").value = ytFetchResult.likes;
            document.getElementById("song-
comments").value = ytFetchResult.comments;
            document.getElementById("song-
thumbnail-url").value =
ytFetchResult.thumbnail;
            const preview =
document.getElementById("song-thumb-
preview");
            preview.src =
ytFetchResult.thumbnail; preview.hidden =
false;
            if (!document.getElementById("song-
label").value) {
                document.getElementById("song-
label").value = ytFetchResult.channelTitle;
            }
            status.textContent = "Details
fetched from YouTube ✓";
            status.className = "is-success";
```

```

        const used =
        parseInt(sessionStorage.getItem("hvc_admin_
        yt_units") || "0", 10);

        sessionStorage.setItem("hvc_admin_yt_units"
        , String(used + 1));
        } catch (err) {
            status.textContent = err.message;
            status.className = "is-error";
        }
    });

    document.getElementById("song-
    thumbnail-url").addEventListener("input",
    (e) => {
        const url = e.target.value.trim();
        const preview =
        document.getElementById("song-thumb-
        preview");
        if (url) { preview.src = url;
        preview.hidden = false; } else {
        preview.hidden = true; }
    });

    document.getElementById("song-thumb-
    file").addEventListener("change", (e) => {
        const file = e.target.files[0];
        if (!file) return;
        const preview =
        document.getElementById("song-thumb-
        preview");
        preview.src =
        URL.createObjectURL(file);
        preview.hidden = false;
    });

```

```
});

document.getElementById("song-
form").addEventListener("submit", async (e)
=> {
    e.preventDefault();
    const saveBtn =
document.getElementById("save-song-btn");
    saveBtn.disabled = true;
    saveBtn.textContent = "Saving...";

    try {
        let thumbnailUrl =
document.getElementById("song-thumbnail-
url").value;
        const file =
document.getElementById("song-thumb-
file").files[0];
        if (file) {
            thumbnailUrl = await
Storage.uploadImage(file, "songs");
        }

        const artistSelect =
document.getElementById("song-artist");
        const artistOption =
artistSelect.selectedOptions[0];

        const data = {
            title:
document.getElementById("song-
title").value.trim(),
            youtubeUrl:
document.getElementById("song-youtube-
```



```
url").value.trim(),
        youtubeVideoId:
ytFetchResult?.videoId ||
YT.extractVideoId(document.getElementById("
song-youtube-url").value),
        formattedDuration:
document.getElementById("song-
duration").value,
        artistId: artistSelect.value ||
null,
        artistName:
artistOption?.dataset.name || "",
        label:
document.getElementById("song-
label").value.trim(),
        genre:
document.getElementById("song-
genre").value,
        releaseDate:
document.getElementById("song-release-
date").value || null,
        rankOverride:
document.getElementById("song-rank-
override").value
        ?
parseInt(document.getElementById("song-
rank-override").value, 10) : null,
        featured:
document.getElementById("song-
featured").checked,
        trending:
document.getElementById("song-
trending").checked,
        views:
```

```

parseInt(document.getElementById("song-
views").value || 0, 10),
    likes:
parseInt(document.getElementById("song-
likes").value || 0, 10),
    comments:
parseInt(document.getElementById("song-
comments").value || 0, 10),
    thumbnail: thumbnailUrl,
    uploadedAt:
ytFetchResult?.uploadedAt || new
Date().toISOString(),
    };

    const id =
document.getElementById("song-id").value;
    if (id) {
        await DB.updateSong(id, data);
        showToast("Song updated.");
    } else {
        await DB.addSong(data);
        showToast("Song added.");
    }
    closeModal("song-modal");
    await loadSongs();
} catch (err) {
    showToast(err.message || "Could not
save song.", true);
} finally {
    saveBtn.disabled = false;
    saveBtn.textContent = "Save Song";
}
});
}

```

```
    async function openSongModal(id) {
      await ensureArtistOptions();
      ytFetchResult = null;
      const form =
document.getElementById("song-form");
      form.reset();
      document.getElementById("song-thumb-
preview").hidden = true;
      document.getElementById("fetch-
status").textContent = "";
      document.getElementById("song-
id").value = id || "";
      document.getElementById("song-modal-
title").textContent = id ? "Edit Song" :
      "Add Song";

      if (id) {
        const song = songsCache.find(s =>
s.id === id) || await DB.getSong(id);
        document.getElementById("song-
youtube-url").value = song.youtubeUrl ||
        "";
        document.getElementById("song-
title").value = song.title || "";
        document.getElementById("song-
duration").value = song.formattedDuration
        || "";
        document.getElementById("song-
artist").value = song.artistId || "";
        document.getElementById("song-
label").value = song.label || "";
        document.getElementById("song-
genre").value = song.genre || "Haryanvi
```

```

Pop";
    document.getElementById("song-
release-date").value = song.releaseDate ||
"";
    document.getElementById("song-rank-
override").value = song.rankOverride || "";
    document.getElementById("song-
featured").checked = !!song.featured;
    document.getElementById("song-
trending").checked = !!song.trending;
    document.getElementById("song-
views").value = song.views || 0;
    document.getElementById("song-
likes").value = song.likes || 0;
    document.getElementById("song-
comments").value = song.comments || 0;
    document.getElementById("song-
thumbnail-url").value = song.thumbnail ||
"";
    if (song.thumbnail) {
        const preview =
document.getElementById("song-thumb-
preview");
        preview.src = song.thumbnail;
        preview.hidden = false;
    }
    }
    openModal("song-modal");
}

/* =====
ARTISTS =====
*/

```

```

    async function loadArtists() {
        artistsCache = await DB.getArtists();
        const grid =
document.getElementById("artists-grid");
        document.getElementById("artists-
empty").hidden = artistsCache.length > 0;
        grid.innerHTML = artistsCache.map(a =>
`
            <div class="artist-tile" data-
id="${a.id}">
                
                <div class="tile-
name">${escapeHtml(a.name)} ${a.verified ?
'<span class="badge badge-
verified">✓</span>' : ""}</div>
                <div class="tile-
meta">${fmtNum(a.monthlyViews)} monthly
views</div>
                <div class="tile-actions">
                    <button class="edit-artist-
btn">Edit</button>
                    <button class="delete-artist-
btn">Delete</button>
                </div>
            </div>
`
        ).join("");

        grid.querySelectorAll(".edit-artist-
btn").forEach(btn => {
            btn.addEventListener("click", (e) =>
openArtistModal(e.target.closest(".artist-
tile").dataset.id));
        });
    }

```

```

        grid.querySelectorAll(".delete-artist-
btn").forEach(btn => {
            btn.addEventListener("click", (e) =>
            {
                const id =
e.target.closest(".artist-
tile").dataset.id;
                const artist = artistsCache.find(a
=> a.id === id);
                confirmDelete("artist", id, `Delete
artist "${artist?.name}"? Their songs will
keep the artist name as text.`);
            });
        });

        function initArtistForm() {
            document.getElementById("add-artist-
btn").addEventListener("click", () =>
openArtistModal(null));

            document.getElementById("artist-avatar-
file").addEventListener("change", (e) => {
                const file = e.target.files[0];
                if (!file) return;
                const preview =
document.getElementById("artist-avatar-
preview");
                preview.src =
URL.createObjectURL(file);
                preview.hidden = false;
            });

            document.getElementById("artist-

```

```

form").addEventListener("submit", async (e)
=> {
    e.preventDefault();
    const saveBtn =
document.getElementById("save-artist-btn");
    saveBtn.disabled = true;
    try {
        let avatarUrl =
document.getElementById("artist-avatar-
url").value;
        const file =
document.getElementById("artist-avatar-
file").files[0];
        if (file) avatarUrl = await
Storage.uploadImage(file, "artists");

        const data = {
            name:
document.getElementById("artist-
name").value.trim(),
            channelUrl:
document.getElementById("artist-channel-
url").value.trim(),
            instagram:
document.getElementById("artist-
instagram").value.trim(),
            bio:
document.getElementById("artist-
bio").value.trim(),
            monthlyViews:
parseInt(document.getElementById("artist-
monthly-views").value || 0, 10),
            verified:
document.getElementById("artist-

```

```

verified").checked,
    avatar: avatarUrl,
  };

  const id =
document.getElementById("artist-id").value;
  if (id) { await DB.updateArtist(id,
data); showToast("Artist updated."); }
  else { await DB.addArtist(data);
showToast("Artist added."); }

  closeModal("artist-modal");
  await loadArtists();
} catch (err) {
  showToast(err.message || "Could not
save artist.", true);
} finally {
  saveBtn.disabled = false;
}
});
}

async function openArtistModal(id) {
  const form =
document.getElementById("artist-form");
  form.reset();
  document.getElementById("artist-avatar-
preview").hidden = true;
  document.getElementById("artist-
id").value = id || "";
  document.getElementById("artist-modal-
title").textContent = id ? "Edit Artist" :
"Add Artist";

```



```

        if (id) {
            const artist = artistsCache.find(a =>
a.id === id);
            document.getElementById("artist-
name").value = artist.name || "";
            document.getElementById("artist-
channel-url").value = artist.channelUrl ||
"";
            document.getElementById("artist-
instagram").value = artist.instagram || "";
            document.getElementById("artist-
bio").value = artist.bio || "";
            document.getElementById("artist-
monthly-views").value = artist.monthlyViews
|| 0;
            document.getElementById("artist-
verified").checked = !!artist.verified;
            document.getElementById("artist-
avatar-url").value = artist.avatar || "";
            if (artist.avatar) {
                const preview =
document.getElementById("artist-avatar-
preview");
                preview.src = artist.avatar;
                preview.hidden = false;
            }
        }
        openModal("artist-modal");
    }

    /* =====
LABELS =====
*/

```

```

    async function loadLabels() {
        labelsCache = await DB.getLabels();
        const grid =
document.getElementById("labels-grid");
        document.getElementById("labels-
empty").hidden = labelsCache.length > 0;
        grid.innerHTML = labelsCache.map(l => `
            <div class="artist-tile" data-
id="${l.id}">
                
                <div class="tile-
name">${escapeHtml(l.name)}</div>
                <div class="tile-
meta">${escapeHtml((l.description ||
"").slice(0, 40))}</div>
                <div class="tile-actions">
                    <button class="edit-label-
btn">Edit</button>
                    <button class="delete-label-
btn">Delete</button>
                </div>
            </div>
        `).join("");

        grid.querySelectorAll(".edit-label-
btn").forEach(btn => {
            btn.addEventListener("click", (e) =>
openLabelModal(e.target.closest(".artist-
tile").dataset.id));
        });
        grid.querySelectorAll(".delete-label-
btn").forEach(btn => {
            btn.addEventListener("click", (e) =>

```

```

{
    const id =
e.target.closest(".artist-
tile").dataset.id;
    const label = labelsCache.find(l =>
l.id === id);
    confirmDelete("label", id, `Delete
label "${label?.name}"?`);
    });
});
}

function initLabelForm() {
    document.getElementById("add-label-
btn").addEventListener("click", () =>
openLabelModal(null));

    document.getElementById("label-logo-
file").addEventListener("change", (e) => {
        const file = e.target.files[0];
        if (!file) return;
        const preview =
document.getElementById("label-logo-
preview");
        preview.src =
URL.createObjectURL(file);
        preview.hidden = false;
    });

    document.getElementById("label-
form").addEventListener("submit", async (e)
=> {
        e.preventDefault();
        const saveBtn =

```

```
document.getElementById("save-label-btn");
    saveBtn.disabled = true;
    try {
        let logoUrl =
document.getElementById("label-logo-
url").value;
        const file =
document.getElementById("label-logo-
file").files[0];
        if (file) logoUrl = await
Storage.uploadImage(file, "labels");

        const data = {
            name:
document.getElementById("label-
name").value.trim(),
            description:
document.getElementById("label-
description").value.trim(),
            website:
document.getElementById("label-
website").value.trim(),
            youtube:
document.getElementById("label-
youtube").value.trim(),
            logo: logoUrl,
        };

        const id =
document.getElementById("label-id").value;
        if (id) { await DB.updateLabel(id,
data); showToast("Label updated."); }
        else { await DB.addLabel(data);
showToast("Label added."); }
```

```

        closeModal("label-modal");
        await loadLabels();
    } catch (err) {
        showToast(err.message || "Could not
save label.", true);
    } finally {
        saveBtn.disabled = false;
    }
});
}

async function openLabelModal(id) {
    const form =
document.getElementById("label-form");
    form.reset();
    document.getElementById("label-logo-
preview").hidden = true;
    document.getElementById("label-
id").value = id || "";
    document.getElementById("label-modal-
title").textContent = id ? "Edit Label" :
"Add Label";

    if (id) {
        const label = labelsCache.find(l =>
l.id === id);
        document.getElementById("label-
name").value = label.name || "";
        document.getElementById("label-
description").value = label.description ||
"";
        document.getElementById("label-
website").value = label.website || "";
    }
}

```

```

        document.getElementById("label-
youtube").value = label.youtube || "";
        document.getElementById("label-logo-
url").value = label.logo || "";
        if (label.logo) {
            const preview =
document.getElementById("label-logo-
preview");
            preview.src = label.logo;
preview.hidden = false;
        }
    }
    openModal("label-modal");
}

/* ===== NEWS
===== */

async function loadNews() {
    newsCache = await DB.getNews();
    const list =
document.getElementById("news-list");
    document.getElementById("news-
empty").hidden = newsCache.length > 0;
    list.innerHTML = newsCache.map(n => `
        <div class="news-item" data-
id="${n.id}">
            
            <div class="news-item-body">
                <div class="news-item-
title">${escapeHtml(n.title)}</div>
                <div class="news-item-meta">
                    <span class="news-status

```

```

    ${n.status || "published"}">${n.status ||
    "published"}</span>
        <span>${escapeHtml(n.category
    || "General")}</span>
        <span>${escapeHtml(n.author ||
    "")}</span>
        <span>${fmtDate(n.publishedAt)}
    </span>
    </div>
</div>
<div class="row-actions">
    <button class="edit-news-btn"
    title="Edit"><svg viewBox="0 0 24 24"><path
    d="M12 20h9M16.5 3.5a2.1 2.1 0 0 1 3 3L7
    19l-4 1 1-4 12.5-12.5Z"
    stroke="currentColor" stroke-width="1.6"
    fill="none" stroke-linejoin="round"/></svg>
    </button>
    <button class="delete-news-btn"
    title="Delete"><svg viewBox="0 0 24 24">
    <path d="M3 6h18M8 6V4a2 2 0 0 1 2-2h4a2 2
    0 0 1 2 2v2m2 0v14a2 2 0 0 1-2 2H8a2 2 0 0
    1-2-2V6h12Z" stroke="currentColor" stroke-
    width="1.6" fill="none" stroke-
    linecap="round"/></svg></button>
    </div>
</div>
` ).join("");

    list.querySelectorAll(".edit-news-
    btn").forEach(btn => {
        btn.addEventListener("click", (e) =>
        openNewsModal(e.target.closest(".news-
        item").dataset.id));
    });

```

```

    });
    list.querySelectorAll(".delete-news-
btn").forEach(btn => {
        btn.addEventListener("click", (e) =>
        {
            const id = e.target.closest(".news-
item").dataset.id;
            confirmDelete("news", id, "Delete
this news post?");
        });
    });
}

function initNewsForm() {
    document.getElementById("add-news-
btn").addEventListener("click", () =>
openNewsModal(null));

    // Rich text toolbar → execCommand on
the contenteditable editor
    document.querySelectorAll(".rte-toolbar
button").forEach(btn => {
        btn.addEventListener("click", () => {
            const editor =
document.getElementById("news-content");
            editor.focus();
            if (btn.dataset.cmd ===
"createLink") {
                const url = prompt("Link URL:");
                if (url)
document.execCommand("createLink", false,
url);
            } else if (btn.dataset.value) {

```



```
document.execCommand(btn.dataset.cmd,  
false, btn.dataset.value);  
    } else {  
  
document.execCommand(btn.dataset.cmd,  
false, null);  
    }  
});  
});  
  
    document.getElementById("news-  
status").addEventListener("change", (e) =>  
{  
    const scheduleField =  
document.getElementById("news-publish-at");  
    scheduleField.disabled =  
e.target.value !== "scheduled";  
});  
  
    document.getElementById("news-thumb-  
file").addEventListener("change", (e) => {  
    const file = e.target.files[0];  
    if (!file) return;  
    const preview =  
document.getElementById("news-thumb-  
preview");  
    preview.src =  
URL.createObjectURL(file);  
    preview.hidden = false;  
});  
  
    document.getElementById("news-  
form").addEventListener("submit", async (e)  
=> {
```

```
        e.preventDefault();
        const saveBtn =
document.getElementById("save-news-btn");
        saveBtn.disabled = true;
        try {
            let thumbnailUrl =
document.getElementById("news-thumbnail-
url").value;
            const file =
document.getElementById("news-thumb-
file").files[0];
            if (file) thumbnailUrl = await
Storage.uploadImage(file, "news");

            const status =
document.getElementById("news-
status").value;
            const scheduledAt =
document.getElementById("news-publish-
at").value;

            const data = {
                title:
document.getElementById("news-
title").value.trim(),
                author:
document.getElementById("news-
author").value.trim() || "Harrywood Charts
Desk",
                category:
document.getElementById("news-
category").value,
                content:
document.getElementById("news-
```

```

content").innerHTML,
        status,
        thumbnail: thumbnailUrl,
        publishedAt: status ===
"scheduled" && scheduledAt ? new
Date(scheduledAt).toISOString() : new
Date().toISOString(),
    };

    const id =
document.getElementById("news-id").value;
    if (id) { await DB.updateNews(id,
data); showToast("News updated."); }
    else { await DB.addNews(data);
showToast("News published."); }

    closeModal("news-modal");
    await loadNews();
    } catch (err) {
        showToast(err.message || "Could not
save news.", true);
    } finally {
        saveBtn.disabled = false;
    }
    });
}

function openNewsModal(id) {
    const form =
document.getElementById("news-form");
    form.reset();
    document.getElementById("news-
content").innerHTML = "";
    document.getElementById("news-thumb-

```

```
preview").hidden = true;
    document.getElementById("news-
id").value = id || "";
    document.getElementById("news-modal-
title").textContent = id ? "Edit News" :
"Add News";
    document.getElementById("news-publish-
at").disabled = true;

    if (id) {
        const news = newsCache.find(n => n.id
=== id);
        document.getElementById("news-
title").value = news.title || "";
        document.getElementById("news-
author").value = news.author || "";
        document.getElementById("news-
category").value = news.category ||
"General";
        document.getElementById("news-
content").innerHTML = news.content || "";
        document.getElementById("news-
status").value = news.status ||
"published";
        document.getElementById("news-
publish-at").disabled = news.status !==
"scheduled";
        document.getElementById("news-
thumbnail-url").value = news.thumbnail ||
"";
        if (news.thumbnail) {
            const preview =
document.getElementById("news-thumb-
preview");
```

```

        preview.src = news.thumbnail;
    preview.hidden = false;
    }
    }
    openModal("news-modal");
}

/* ===== TOP
100 ===== */

function initTop100() {
    document.getElementById("add-to-top100-
btn").addEventListener("click",
openPicker);
    document.getElementById("save-ranks-
btn").addEventListener("click",
saveRankOrder);
    document.getElementById("auto-rank-
btn").addEventListener("click",
autoRankByViews);
    document.getElementById("archive-
btn").addEventListener("click",
archiveWeek);
    document.getElementById("lock-
btn").addEventListener("click",
toggleLock);

    document.getElementById("picker-
search").addEventListener("input", (e) =>
renderPickerList(e.target.value));
}

async function loadTop100() {
    songsCache = await DB.getSongs();

```

```

    top100Meta = await DB.getTop100Meta();
    renderRankList();
    renderArchiveTable();
    updateLockUI();
  }

  function renderRankList() {
    const ranked = songsCache
      .filter(s => s.weeklyRank)
      .sort((a, b) => a.weeklyRank -
b.weeklyRank);

    const list =
document.getElementById("rank-list");
    document.getElementById("top100-
empty").hidden = ranked.length > 0;
    list.classList.toggle("is-locked",
top100Meta.locked);

    list.innerHTML = ranked.map((s, i) => `
      <li class="rank-row"
draggable="${!top100Meta.locked}" data-
id="${s.id}">
        <span class="drag-handle">⋮</span>
        <div class="rank-num"><input
type="number" min="1" max="100" value="${i
+ 1}" data-id="${s.id}" ${top100Meta.locked
? "disabled" : ""}></div>
        
        <div>
          <div class="rank-
title">${escapeHtml(s.title)}</div>
          <div class="rank-

```

```

artist">${escapeHtml(s.artistName || "")}
</div>
    </div>
    <button class="rank-remove"
title="Remove from chart"
${top100Meta.locked ? "disabled" : ""}><svg
viewBox="0 0 24 24"><path d="M18 6 6 18M6
6 12 12" stroke="currentColor" stroke-
width="2" fill="none" stroke-
linecap="round"/></svg></button>
    </li>
    `).join("");

    if (top100Meta.locked) return;

    // Drag-and-drop reordering
    list.querySelectorAll(".rank-
row").forEach(row => {
        row.addEventListener("dragstart", ()
=> { draggedRow = row;
row.classList.add("is-dragging"); });
        row.addEventListener("dragend", () =>
{ row.classList.remove("is-dragging"); });
        row.addEventListener("dragover", (e)
=> { e.preventDefault();
row.classList.add("is-over"); });
        row.addEventListener("dragleave", ()
=> row.classList.remove("is-over"));
        row.addEventListener("drop", (e) => {
            e.preventDefault();
            row.classList.remove("is-over");
            if (draggedRow && draggedRow !==
row) {
                const rows = [...list.children];

```

```

        const from =
rows.indexOf(draggedRow);
        const to = rows.indexOf(row);
        if (from < to)
row.after(draggedRow); else
row.before(draggedRow);
        renumberRankInputs();
    }
    });
});

list.querySelectorAll(".rank-
remove").forEach(btn => {
    btn.addEventListener("click", async
(e) => {
        const id = e.target.closest(".rank-
row").dataset.id;
        await DB.clearWeeklyRank(id);
        showToast("Removed from Weekly Top
100.");
        await loadTop100();
    });
});
}

function renumberRankInputs() {
    document.querySelectorAll("#rank-list
.rank-row").forEach((row, i) => {
        row.querySelector("input").value = i
+ 1;
    });
}

async function saveRankOrder() {

```



```

    if (top100Meta.locked) {
      showToast("Chart is locked. Unlock it
first.", true); return; }
    const rows =
[...document.querySelectorAll("#rank-list
.rank-row")];
    // Respect any manually typed rank
numbers by sorting on them first
    const withRanks = rows.map(row => ({
      id: row.dataset.id,
      rank:
parseInt(row.querySelector("input").value,
10) || 999,
    })).sort((a, b) => a.rank - b.rank);

    try {
      await
DB.saveWeeklyRanks(withRanks.map(r =>
r.id));
      showToast("Weekly Top 100 order
saved.");
      await loadTop100();
    } catch (err) {
      showToast(err.message || "Could not
save order.", true);
    }
  }

  async function autoRankByViews() {
    if (top100Meta.locked) {
      showToast("Chart is locked. Unlock it
first.", true); return; }
    const sorted = [...songsCache]
      .sort((a, b) => (b.rankOverride ? -1

```

```

: 0) - (a.rankOverride ? -1 : 0) ||
(b.views || 0) - (a.views || 0))
    .slice(0, 100);
    try {
        await DB.saveWeeklyRanks(sorted.map(s
=> s.id));
        showToast("Auto-ranked by views.");
        await loadTop100();
    } catch (err) {
        showToast(err.message || "Auto-rank
failed.", true);
    }
}

async function toggleLock() {
    top100Meta.locked = !top100Meta.locked;
    await
DB.setTop100Locked(top100Meta.locked);
    updateLockUI();
    renderRankList();
    showToast(top100Meta.locked ? "Chart
locked." : "Chart unlocked.");
}

function updateLockUI() {
    const btn =
document.getElementById("lock-btn");
    btn.textContent = top100Meta.locked ?
"🔒 Locked" : "🔓 Unlocked";
    btn.classList.toggle("is-locked",
top100Meta.locked);
    document.getElementById("save-ranks-
btn").disabled = top100Meta.locked;
    document.getElementById("auto-rank-

```

```

btn").disabled = top100Meta.locked;
}

async function archiveWeek() {
  try {
    const weekId = await
DB.archiveCurrentTop100(songsCache);
    showToast(`Archived as ${weekId}.`);
    renderArchiveTable();
  } catch (err) {
    showToast(err.message || "Archive
failed.", true);
  }
}

async function renderArchiveTable() {
  const archives = await
DB.getArchives();
  const tbody =
document.getElementById("archive-
table").querySelector("tbody");
  tbody.innerHTML = archives.map(a => `
    <tr><td>${a.weekId}</td>
<td>${a.songs?.length || 0} songs</td>
<td>${fmtDate(a.archivedAt)}</td></tr>
  `).join("") || `<tr><td colspan="3"
class="empty-state">No archived weeks yet.
</td></tr>`;
}

function openPicker() {
  document.getElementById("picker-
search").value = "";
  renderPickerList("");
}

```

```

    openModal("picker-modal");
  }

  function renderPickerList(query) {
    const unranked = songsCache.filter(s =>
!s.weeklyRank);
    const filtered =
DB.searchSongs(unranked, query);
    const list =
document.getElementById("picker-list");
    list.innerHTML = filtered.slice(0,
50).map(s => `
      <li class="picker-item" data-
id="${s.id}">
        
        <span class="picker-item-
title">${escapeHtml(s.title)} –
${escapeHtml(s.artistName || "")}</span>
        <button data-
id="${s.id}">Add</button>
      </li>
    `).join("") || `<li class="empty-
state">No matching songs.</li>`;

    list.querySelectorAll("button[data-
id]").forEach(btn => {
      btn.addEventListener("click", async
() => {
        const currentMax =
songsCache.filter(s =>
s.weeklyRank).length;
        await
window.HVC.db.collection("songs").doc(btn.d

```

```

ataset.id).update({ weeklyRank: currentMax
+ 1 });
    showToast("Added to Weekly Top
100.");
    closeModal("picker-modal");
    await loadTop100();
  });
});
}

/* =====
SETTINGS
===== */

function initSettingsForm() {
  ["logo", "favicon"].forEach(kind => {

document.getElementById(`settings-${kind}-
file`).addEventListener("change", (e) => {
  const file = e.target.files[0];
  if (!file) return;
  const preview =
document.getElementById(`settings-${kind}-
preview`);
  preview.src =
URL.createObjectURL(file);
  preview.hidden = false;
});
});

  document.getElementById("settings-
form").addEventListener("submit", async (e)
=> {
    e.preventDefault();

```

```
        try {
            let logoUrl =
document.getElementById("settings-logo-
url").value;
            let faviconUrl =
document.getElementById("settings-favicon-
url").value;
            const logoFile =
document.getElementById("settings-logo-
file").files[0];
            const faviconFile =
document.getElementById("settings-favicon-
file").files[0];
            if (logoFile) logoUrl = await
Storage.uploadImage(logoFile, "settings");
            if (faviconFile) faviconUrl = await
Storage.uploadImage(faviconFile,
"settings");

            await DB.saveSettings({
                siteName:
document.getElementById("settings-site-
name").value.trim(),
                contactEmail:
document.getElementById("settings-contact-
email").value.trim(),
                logo: logoUrl,
                favicon: faviconUrl,
                instagram:
document.getElementById("settings-
instagram").value.trim(),
                youtube:
document.getElementById("settings-
youtube").value.trim(),
```

```

        metaDescription:
document.getElementById("settings-meta-
description").value.trim(),
        metaKeywords:
document.getElementById("settings-meta-
keywords").value.trim(),
    });
    showToast("Settings saved.");
    } catch (err) {
        showToast(err.message || "Could not
save settings.", true);
    }
    });
}

    async function loadSettings() {
        const settings = await
DB.getSettings();
        document.getElementById("settings-site-
name").value = settings.siteName ||
"Harrywood Charts";
        document.getElementById("settings-
contact-email").value =
settings.contactEmail || "";
        document.getElementById("settings-
instagram").value = settings.instagram ||
"";
        document.getElementById("settings-
youtube").value = settings.youtube || "";
        document.getElementById("settings-meta-
description").value =
settings.metaDescription || "";
        document.getElementById("settings-meta-
keywords").value = settings.metaKeywords ||

```

```

"";
    document.getElementById("settings-logo-
url").value = settings.logo || "";
    document.getElementById("settings-
favicon-url").value = settings.favicon ||
"";
    if (settings.logo) { const p =
document.getElementById("settings-logo-
preview"); p.src = settings.logo; p.hidden
= false; }
    if (settings.favicon) { const p =
document.getElementById("settings-favicon-
preview"); p.src = settings.favicon;
p.hidden = false; }
    }

    /* =====
SEARCH =====
*/

    function initSearch() {
        document.getElementById("song-
search").addEventListener("input", (e) => {

renderSongsTable(DB.searchSongs(songsCache,
e.target.value));
        });
    }

})();

```